

Java Project: Student Performance Analyzer with Arrays

Project Title:

"Advanced Student Performance Management System Using 1D and 2D Arrays"

Project Overview:

This project simulates a **full-featured student performance tracker** using one-dimensional and two-dimensional arrays. It includes:

- Dynamic input of students and subjects
- Recording and processing of marks
- Grading system
- Ranking students
- Subject-wise analysis

Designed to strengthen students' understanding of:

- Array traversal
- Algorithmic thinking
- Modular programming
- Nested loops
- Data analysis using arrays

Data Representation:

- **1D Arrays:**
 - studentNames[]: Stores names of students
 - subjectNames[]: Stores subject titles
 - totalMarks[]: Stores total marks per student
 - averageMarks[]: Stores average marks per student
 - grades[]: Stores grade letters (A, B, C...) per student
- **2D Array:**
 - marks[][]: marks[i][j] stores marks of student i in subject j

Project Functionalities:

1. Input Data

- Inputs number of students and subjects
- Inputs student names and subject titles
- Inputs marks of each student per subject

Pseudocode:

```
Function inputMarks()
  For i from 0 to numStudents - 1
    Read studentNames[i]
    For j from 0 to numSubjects - 1
      Read marks[i][j]
```

2. Calculate Totals and Averages

- Sum all subject marks per student
- Store total in totalMarks[i], average in averageMarks[i]

Pseudocode:

```
Function calculateTotalsAndAverages()
  For i from 0 to numStudents - 1
    sum ← 0
    For j from 0 to numSubjects - 1
      sum ← sum + marks[i][j]
    totalMarks[i] ← sum
    averageMarks[i] ← sum / numSubjects
```

3. Assign Grades

- Based on average:
A (90+), B (80–89), C (70–79), D (60–69), F (<60)

Pseudocode:

```
Function assignGrades()
  For i from 0 to numStudents - 1
    If averageMarks[i] >= 90 then grades[i] ← 'A'
    Else if averageMarks[i] >= 80 then grades[i] ← 'B'
    Else if averageMarks[i] >= 70 then grades[i] ← 'C'
    Else if averageMarks[i] >= 60 then grades[i] ← 'D'
    Else grades[i] ← 'F'
```

4. Display Report Table

- Shows names, subject marks, total, average, grade

Pseudocode:

```
Function displayReport()
  Print headers
  For i from 0 to numStudents - 1
    Print studentNames[i]
    For j from 0 to numSubjects - 1
```

```
    Print marks[i][j]
    Print totalMarks[i], averageMarks[i], grades[i]
```

5. Find Topper

- Find the student with the highest total marks

Pseudocode:

```
Function findTopper()
    maxIndex ← 0
    For i from 1 to numStudents - 1
        If totalMarks[i] > totalMarks[maxIndex]
            maxIndex ← i
    Print studentNames[maxIndex] as Topper
```

6. Subject-wise Averages

- Computes and displays average marks for each subject

Pseudocode:

```
Function subjectAverages()
    For j from 0 to numSubjects - 1
        sum ← 0
        For i from 0 to numStudents - 1
            sum ← sum + marks[i][j]
        avg ← sum / numStudents
        Print subjectNames[j], avg
```

7. Sort by Rank

- Sorts students in descending order of totalMarks

Pseudocode:

```
Function sortByRank()
    For i from 0 to numStudents - 2
        For j from i + 1 to numStudents - 1
            If totalMarks[j] > totalMarks[i]
                Swap studentNames[i] and studentNames[j]
                Swap totalMarks[i] and totalMarks[j]
                Swap averageMarks[i] and averageMarks[j]
                Swap grades[i] and grades[j]
                Swap marks[i] and marks[j] // Entire row
```